What’s in the bag? (Random forests)

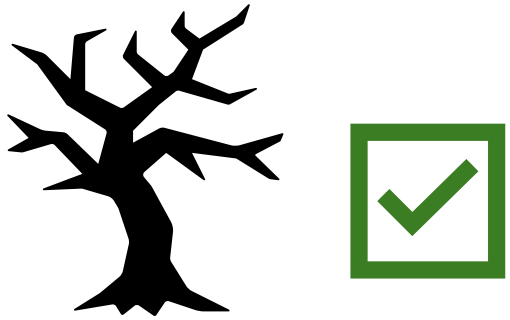
Jeremiah Shrovnal, Grant Vagle, Lynn Waterhouse

Overview of this section

- Bagging
  - Algorithm overview
- Random forest models
  - Algorithm overview
  - Intro to model tuning
- Variable importance - intention and calculation
- Partial dependence - how it's calculated and how to interpret
- Considerations with correlated predictors
- Introduce the coding examples
- Hands-on coding example

CART model approach

- Variance vs bias trade off/balance
- Too much tree results in a lot of variance
- Prune back branches to reduce variance
  - Introduces bias



Reducing variance in our estimates

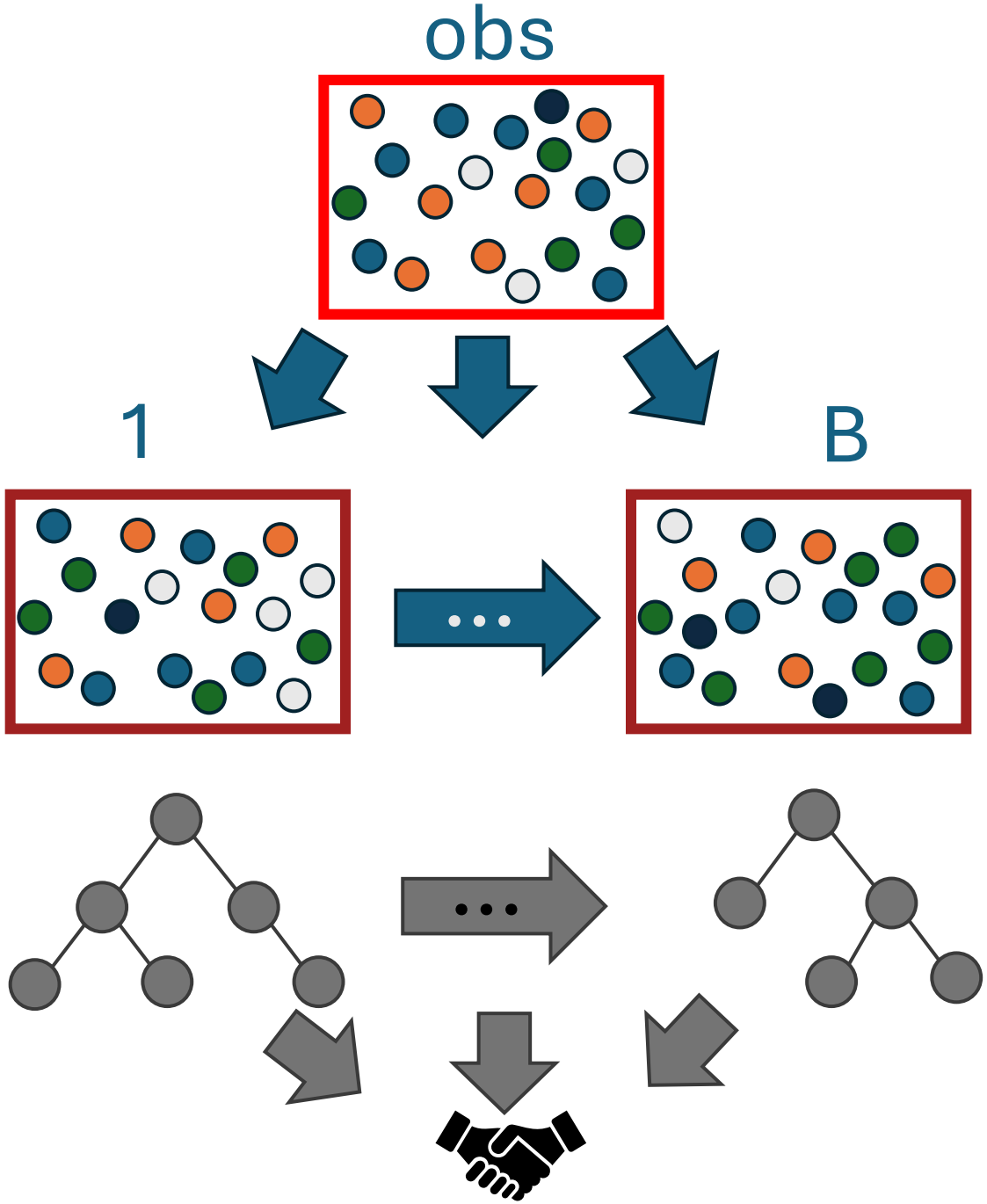
- What if we had many independent observations of what we'd like to model?
- We could grow a tree on each training set and then take the average tree
- Generally, not practical because we won't have access to multiple training sets…

Bagging

- We can bootstrap!
  - Take repeated samples (*with replacement*) from the (single) training set
- This ***b**ootstrap **agg**regation*, known as **bagging**, allows us to reduce variance by producing a whole bunch of trees and averaging over them

Bagging, explained

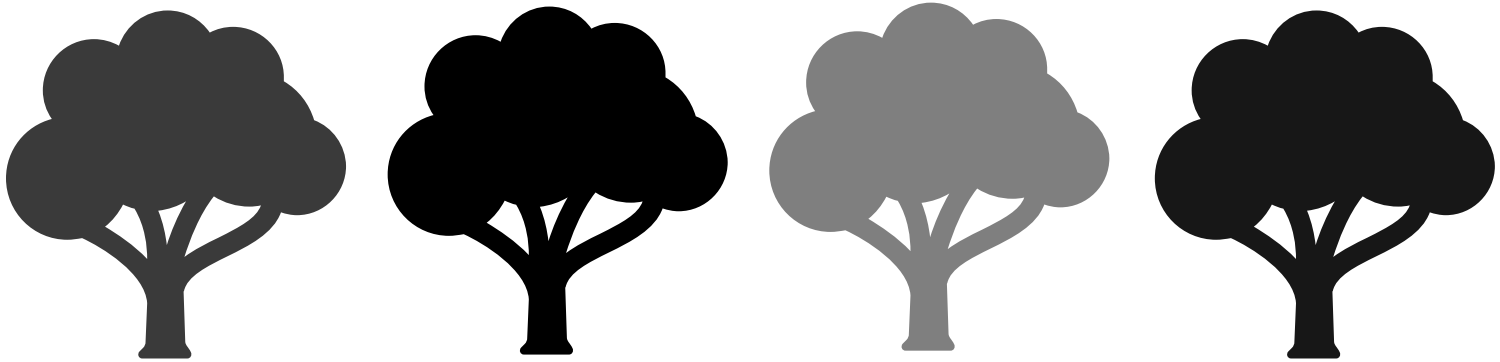
- Generate *B* different bootstrapped training sets, all coming from a single data set, to create *pseudo* training sets
- Train on the *b*th bootstrapped training set to get a prediction
- Average over all of the predictions (regression) or take the majority vote (classification)



Bagging – some new concepts

Previous methods pruned the trees to help reduce variance

- When you prune a tree you introduce bias by making it coarser
- Large bushy trees tend to have less bias but more variance
- Averaging over a bunch of large bushy trees can reduce variance
  - We do not need to worry about pruning the trees with bagging!



Out-of-bag error

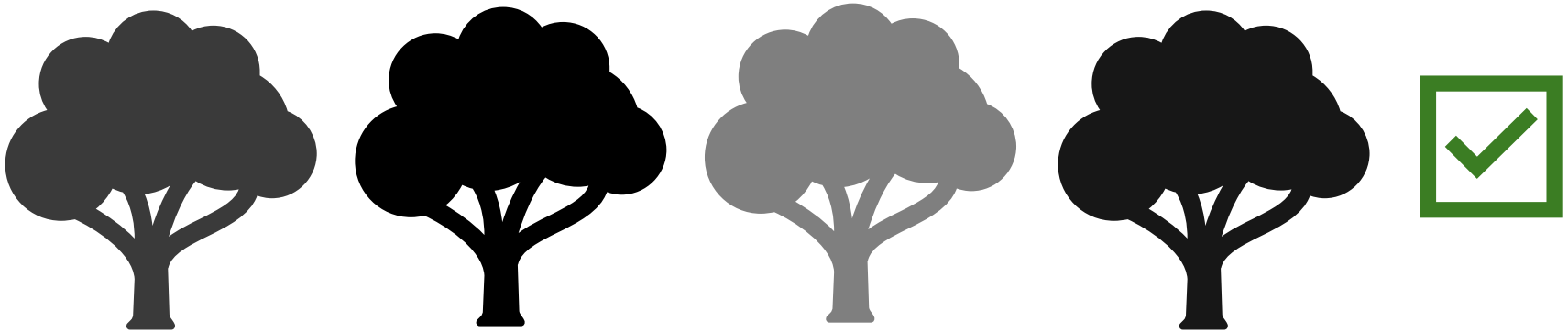
A very nice and straightforward way to estimate test error of a bagged model

- Bagging involves building trees from bootstrapped subsets, which uses about 2/3 of the observations
- Remaining 1/3 not used to fit a given tree is referred to as **out-of-bag** (OOB)
- Can predict the response for OOB observations that we can average, giving around B/3 predictions per observation
- Essentially a free leave-one-out cross-validation

Random forest

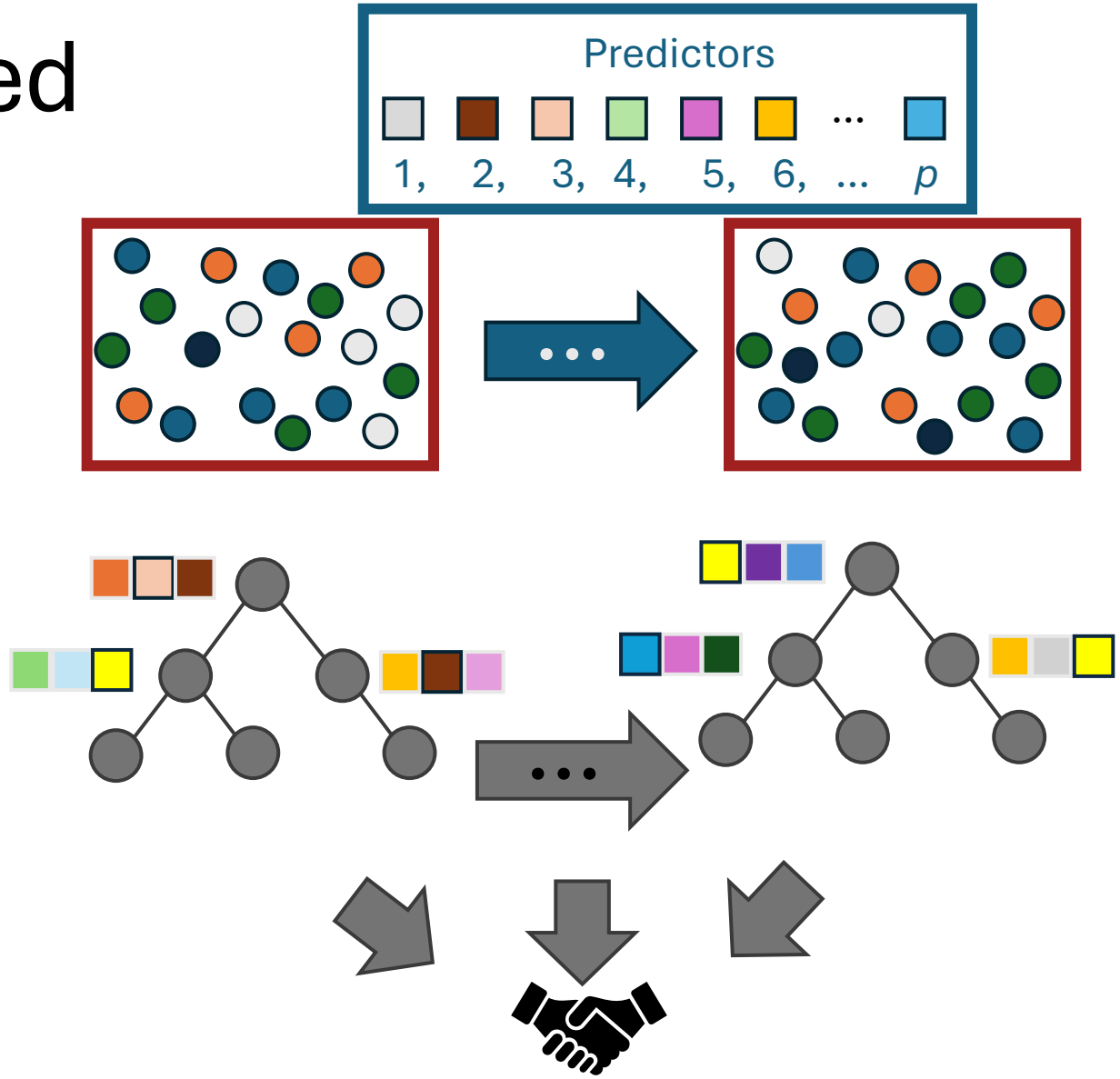
Improvement over bagged trees

- Include a small tweak that *decorrelates* the trees
- This then reduces the variance when we average over lots of bushy trees



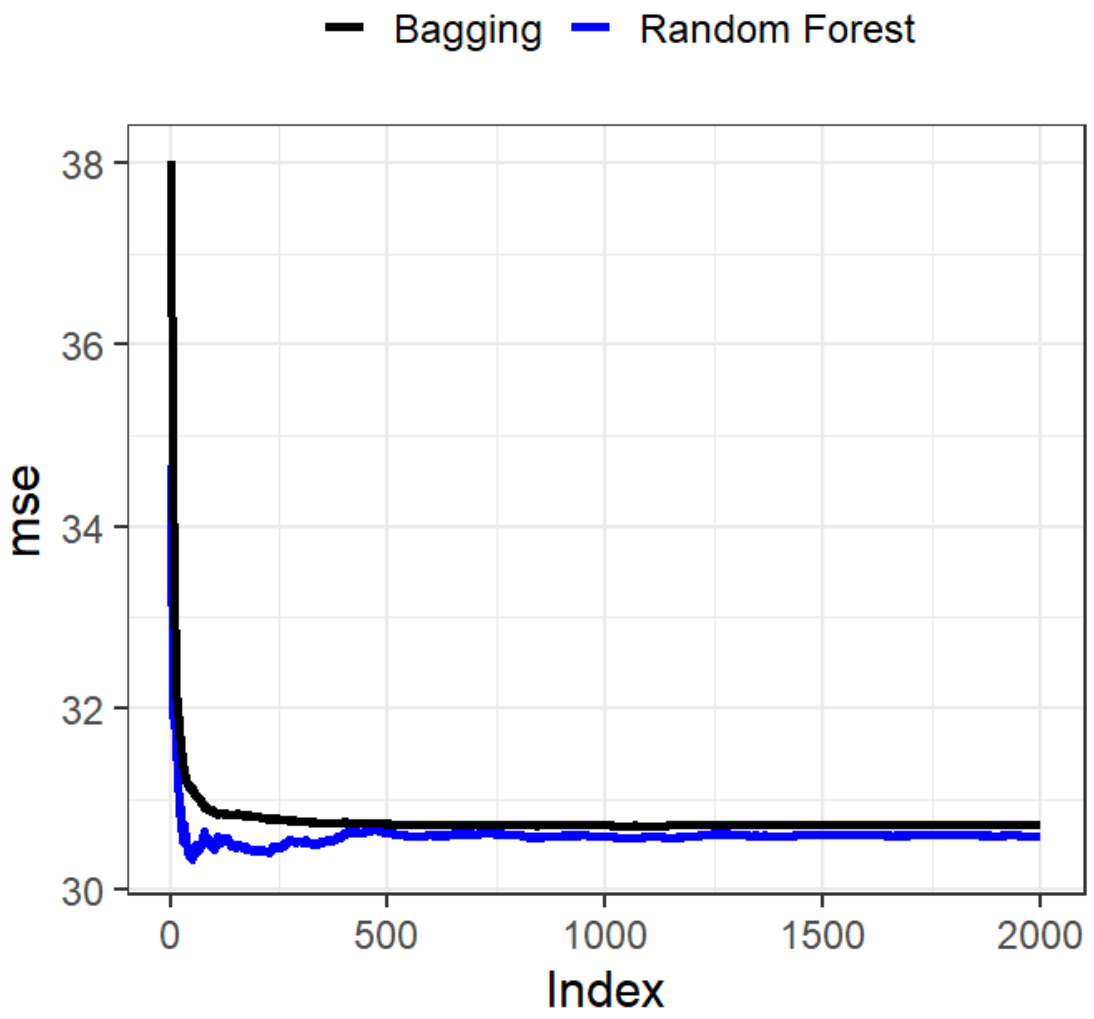
Random forest, explained

- Build a number of decision trees on bootstrapped training samples (as with *bagging*)
- When building a tree, each time a split in the tree is considered: **a random selection of *m* predictors** is chosen as potential split candidates from the full set of *p* predictors
  - The split is allowed to use only one of those *m* predictors
  - A new set of *m* predictors is taken at each split



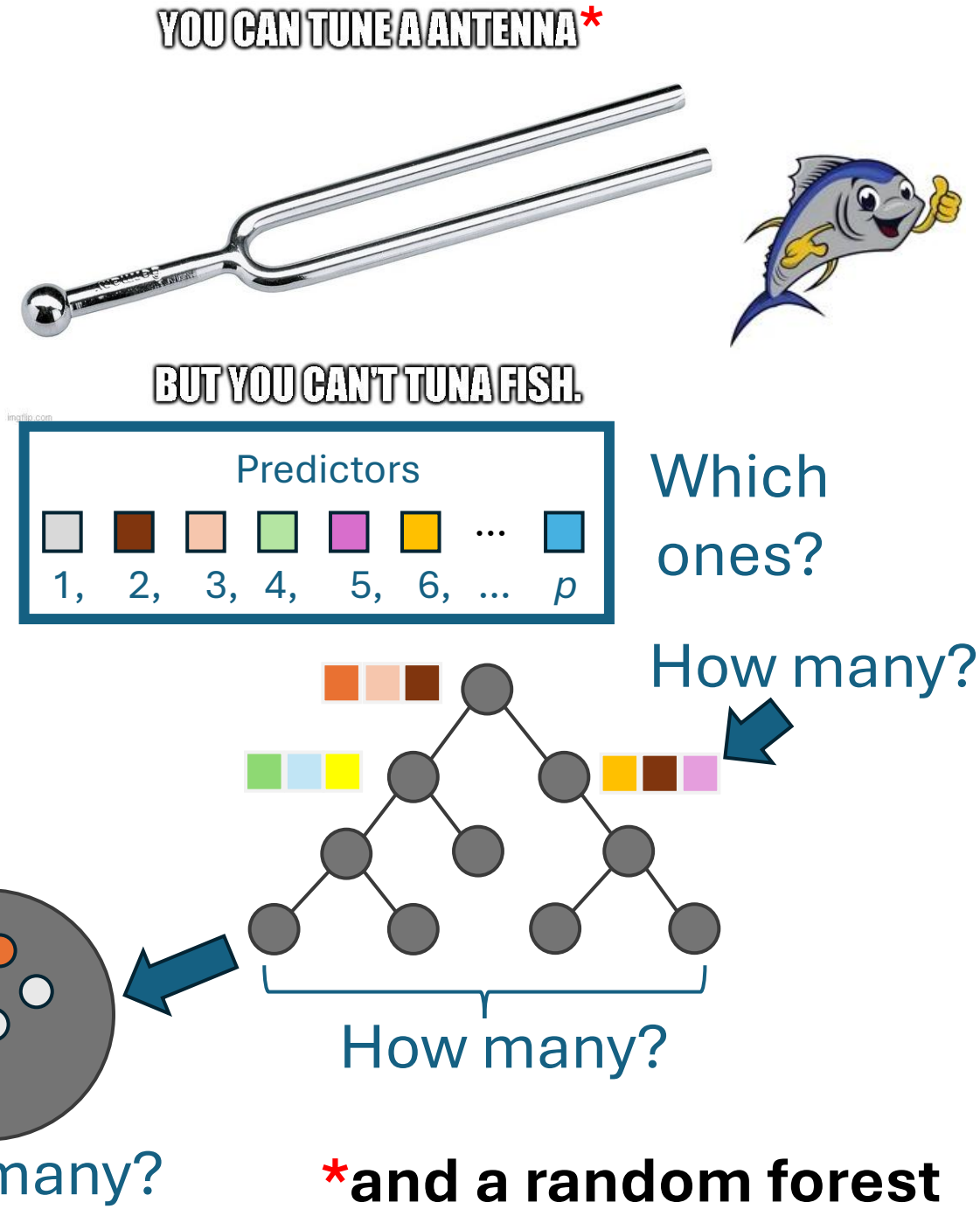
How many trees to plant in the forest?

- A benefit of these two methods is that you **can't** overfit by growing too many trees
- The benefit of adding more trees is it brings the variance down
- At some point the variance will stop decreasing
  - When you reach this point, adding more trees won't help you
  - But it also won't hurt you!



Model tuning

- The ***m*** number of the ***p*** total parameters considered at each split (***mtry*** in the code)
  - Defaults to square-root(***p***) for classification or ***p***/3 for regression
- The size of the tree
  - ***Terminal node size***: minimum size-defaults to 1 for classification or 5 for regression
  - ***Maximum terminal nodes***
- The ***p*** predictors considered

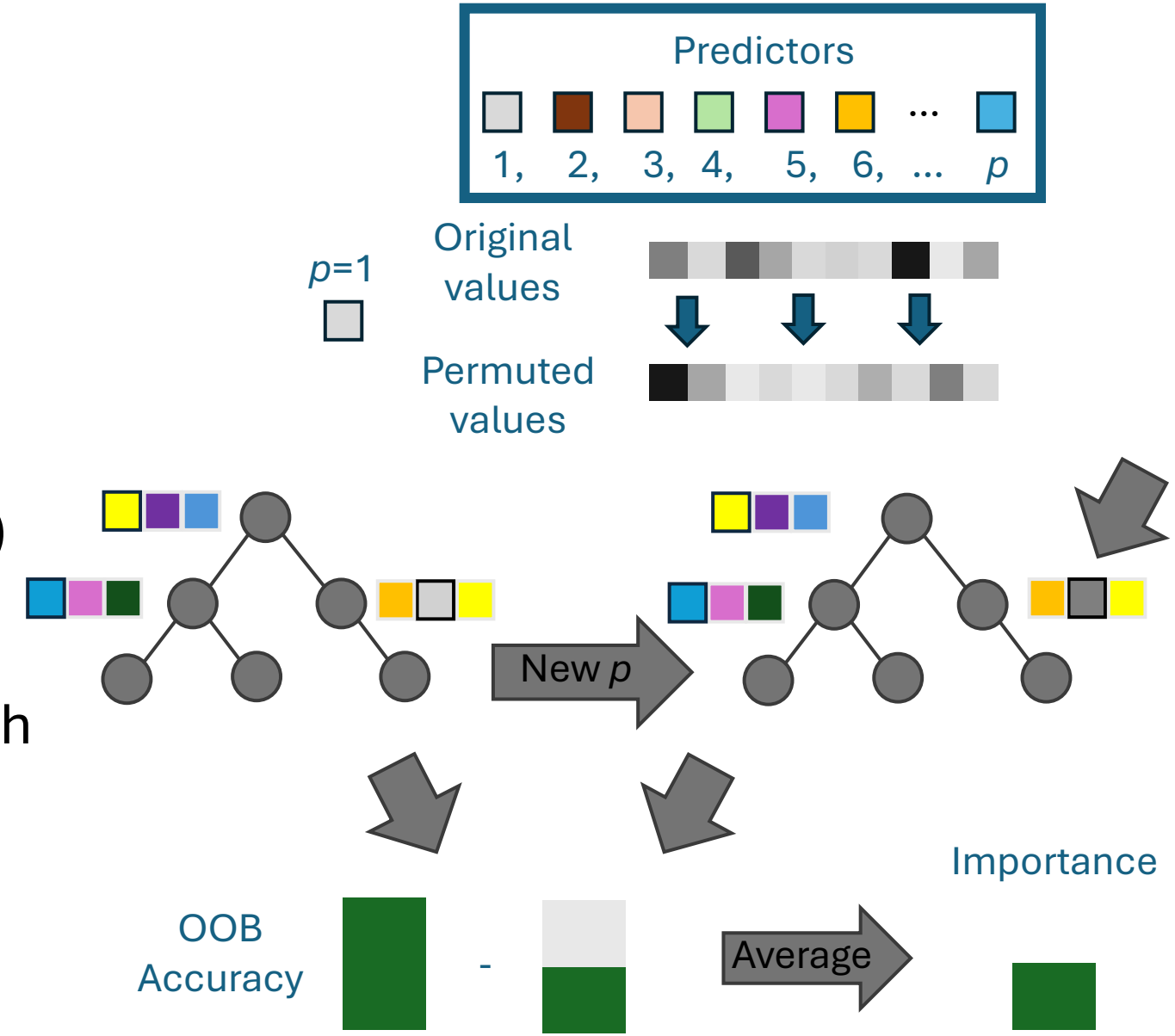


***and a random forest**

Variable importance

Find variables that influence the outcome

- Start with some baseline measure of fit (OOB error)
- Shuffle the value (one at a time) for each of the *p* predictors
- Pass these randomly permuted values of a single *p* back through the tree
- Compare resulting shift in accuracy across forest

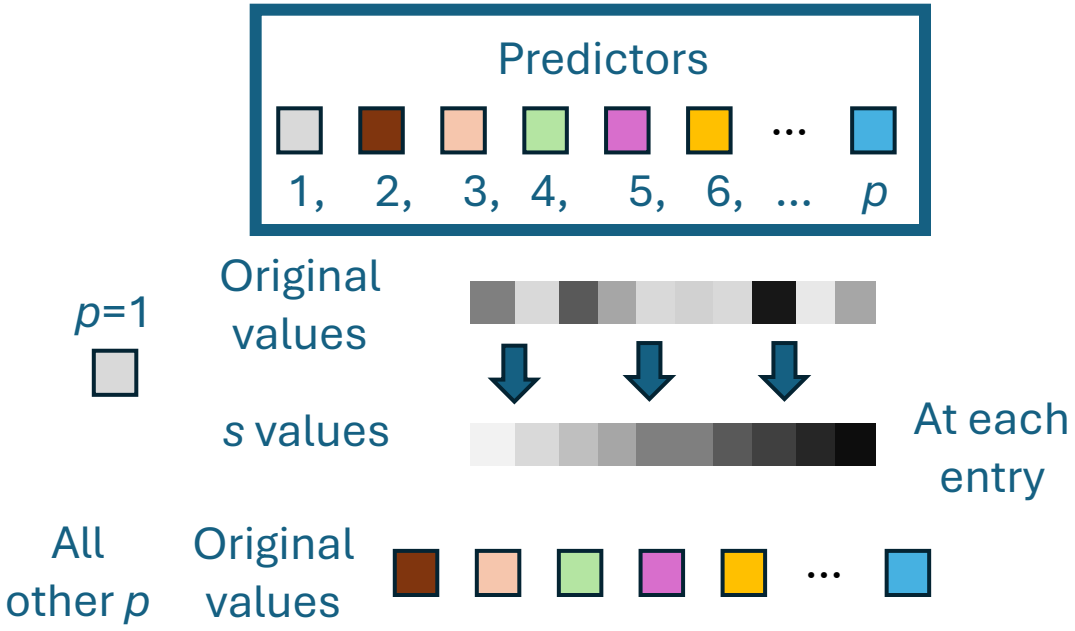


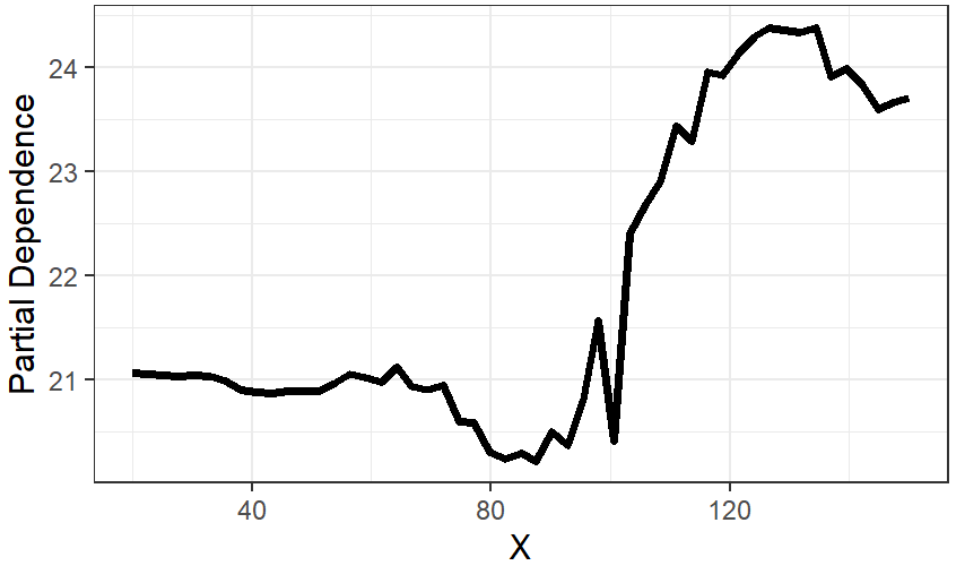
Partial dependence

An attempt to isolate the relationship between the predictor and response

- Fix the values for one of the *p* predictors of interest to a specified value *s* while holding every other value constant
- Predict with the model
- Move *s* across the observed range
- Average the predictions at each *s*

This is the **marginal effect** of the predictor of interest

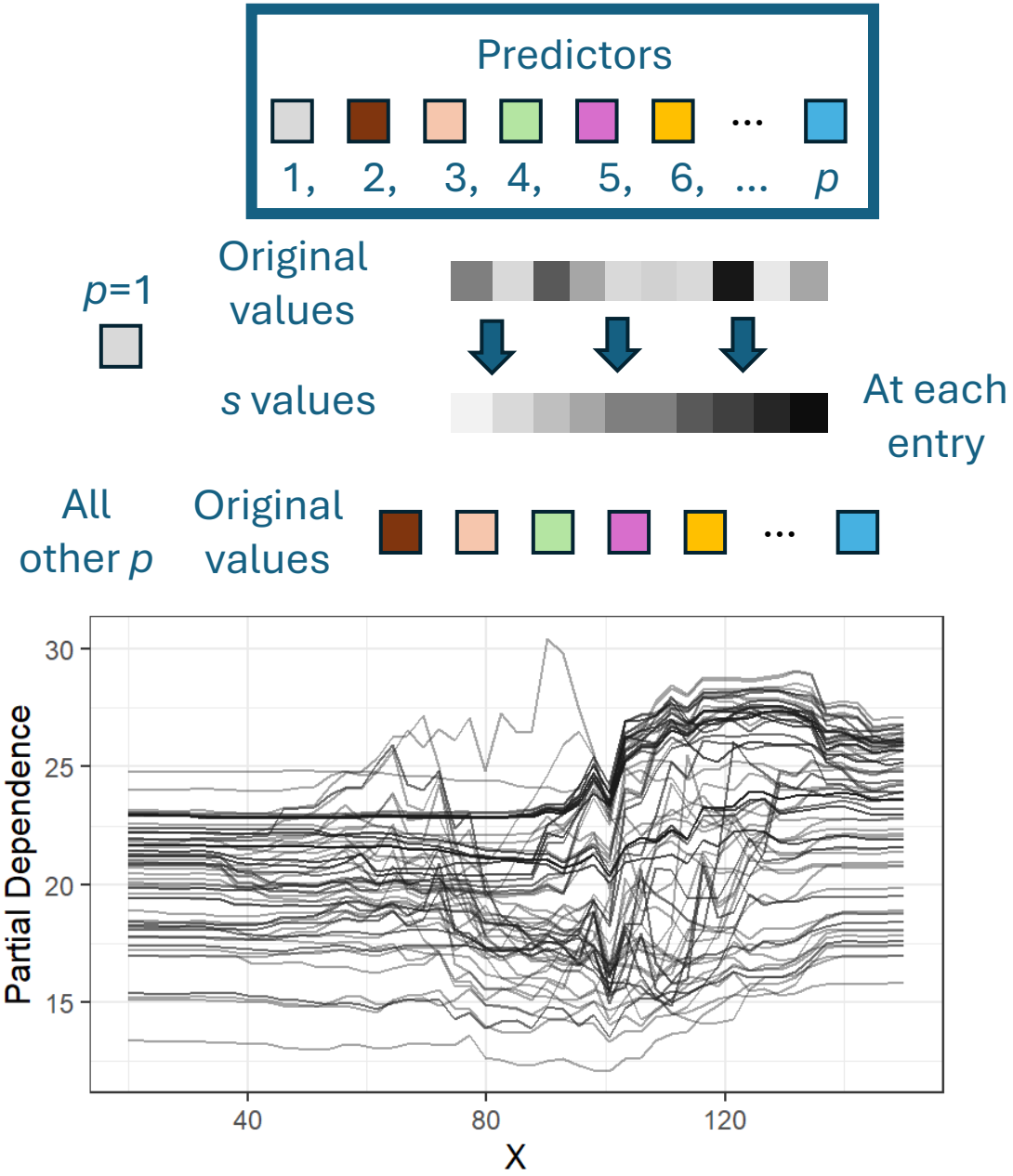




Partial dependence

What if we want to look at each entry?

- This is known as the Individual Conditional Expectation (ICE)
- Can be used to identify heterogeneity in the relationships when curves aren't parallel
- These are what are averaged to create the partial dependence plot (PDP)

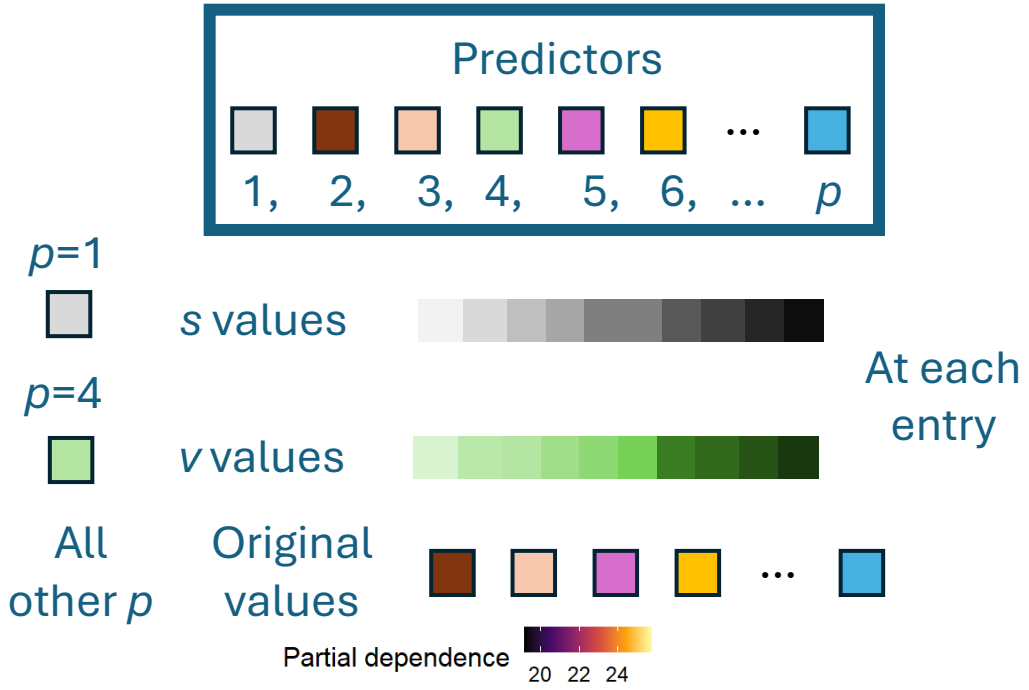


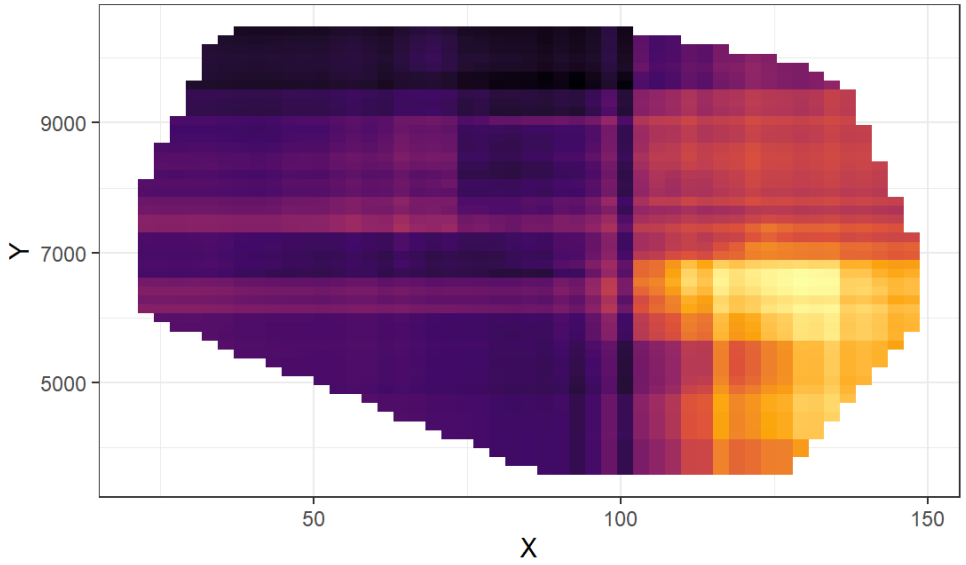
Partial dependence

What if we're interested in the relationship among two predictors and the response

- Fix the values for two of the *p* predictors of interest to specified values *s* and *v* while holding every other value constant
- Predict with the model
- Move *s* and *v* across their observed ranges
- Average the predictions at each *s* and *v*

This is the **marginal effect** of both the predictors of interest





Correlated predictors

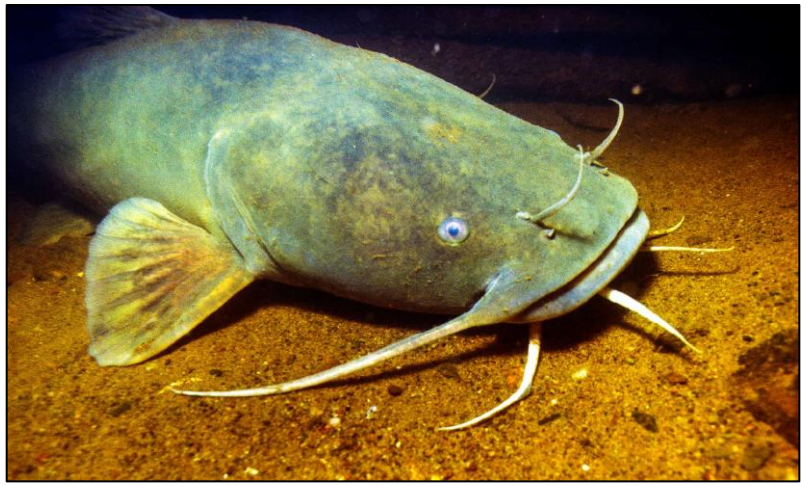
They make interpretation hard!

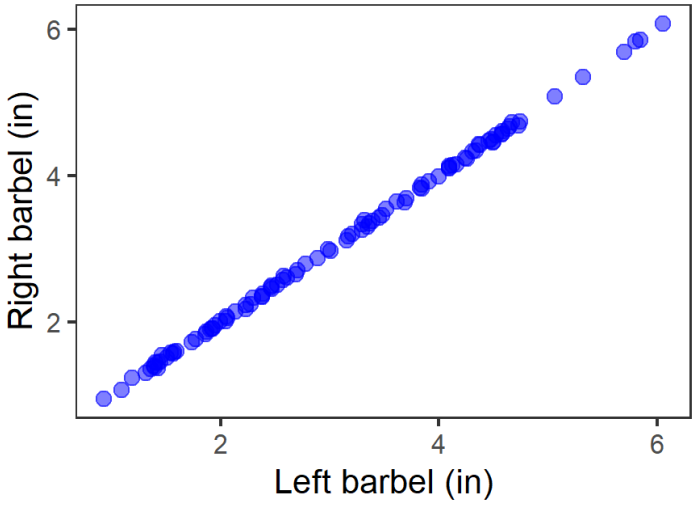
- The raw correlation isn't what matters, but instead the association conditional on the other variables in the model
- There is *nothing wrong* with multicollinearity, and the model will work fine for **prediction** (you'll just be frustrated trying to understand it)
- We'll use a simulation with a linear model here to try to lay out some considerations for dealing with correlated variables

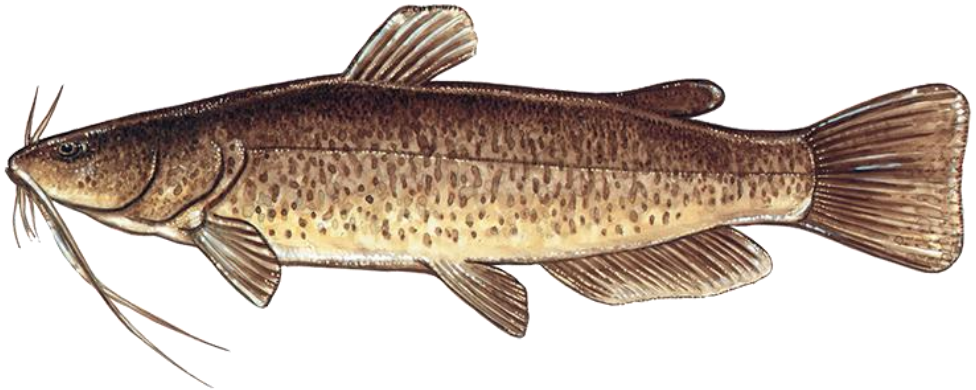
Correlated barbels

Imagine trying to predict a catfish's length based on its longest barbels in a standard regression

- Assume total length has positive association with barbel length
- We have information on both barbels
- What happens if we fit a model with both?







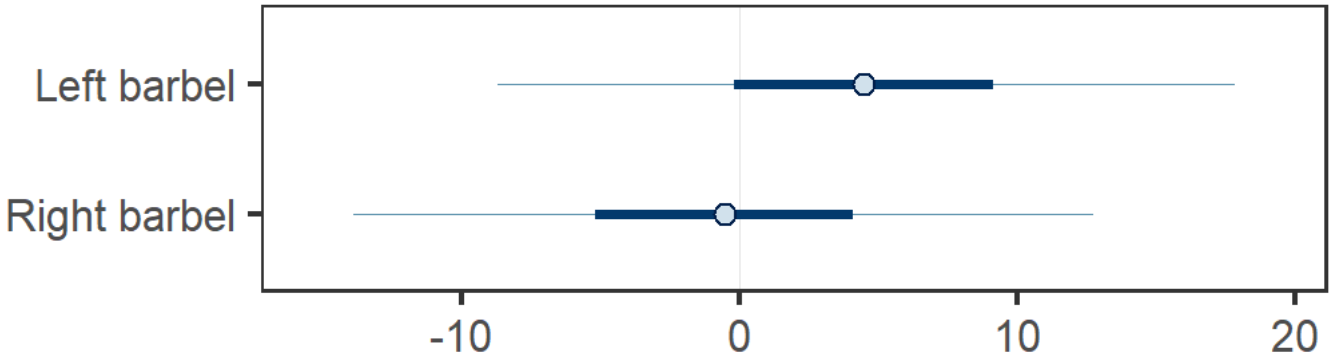
ρ	Left barbel	Right barbel
Left barbel	1.0000	0.9998
Right barbel	0.9998	1.0000

Correlated barbels

$$\text{Total Length}_i \sim Normal(\mu_i, \sigma^2)$$
$$\mu_i = \text{Intercept} + \text{Left barbel}_i + \text{Right barbel}_i$$

It fits no problem…but why is the fit so weird for each parameter? Did it work correctly??

Parameter	Estimate	Est.Error	Q2.5	Q97.5
Left barbel	−0.53	6.82	−13.88	12.73
Right barbel	4.48	6.82	−8.72	17.81



Correlated barbels

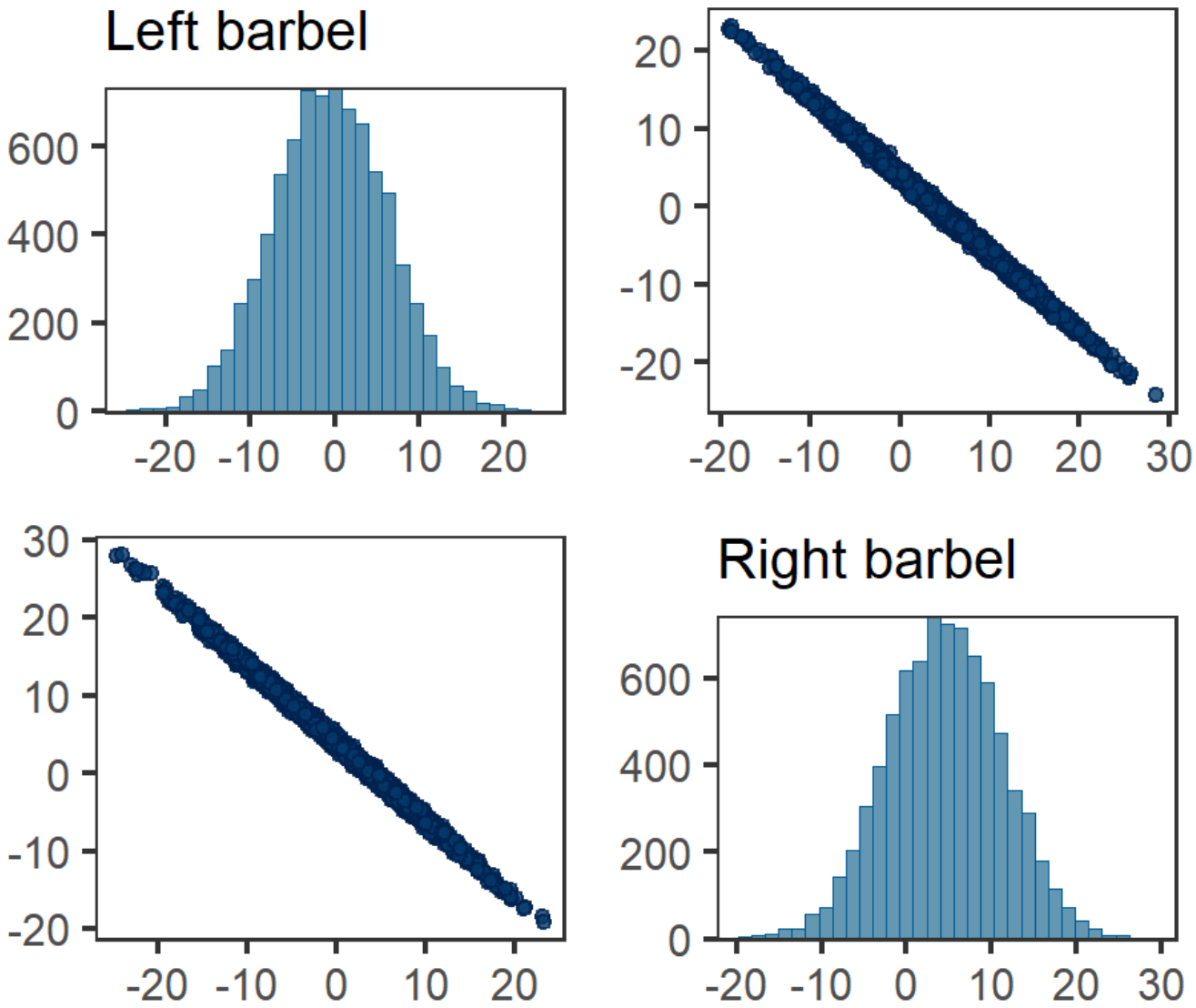
It did work correctly and it is the right answer to the question we asked. **The problem is the question**.

- For this model: *What is the value of knowing each predictor, after already knowing all of the other predictors?*
- So, for this question: *What is the value of knowing each barbel's length, after already knowing the other barbel's length?*
  - What was shown earlier was the distribution considering every possible combination of parameters and assigning relative plausibilities to each

Correlated barbels

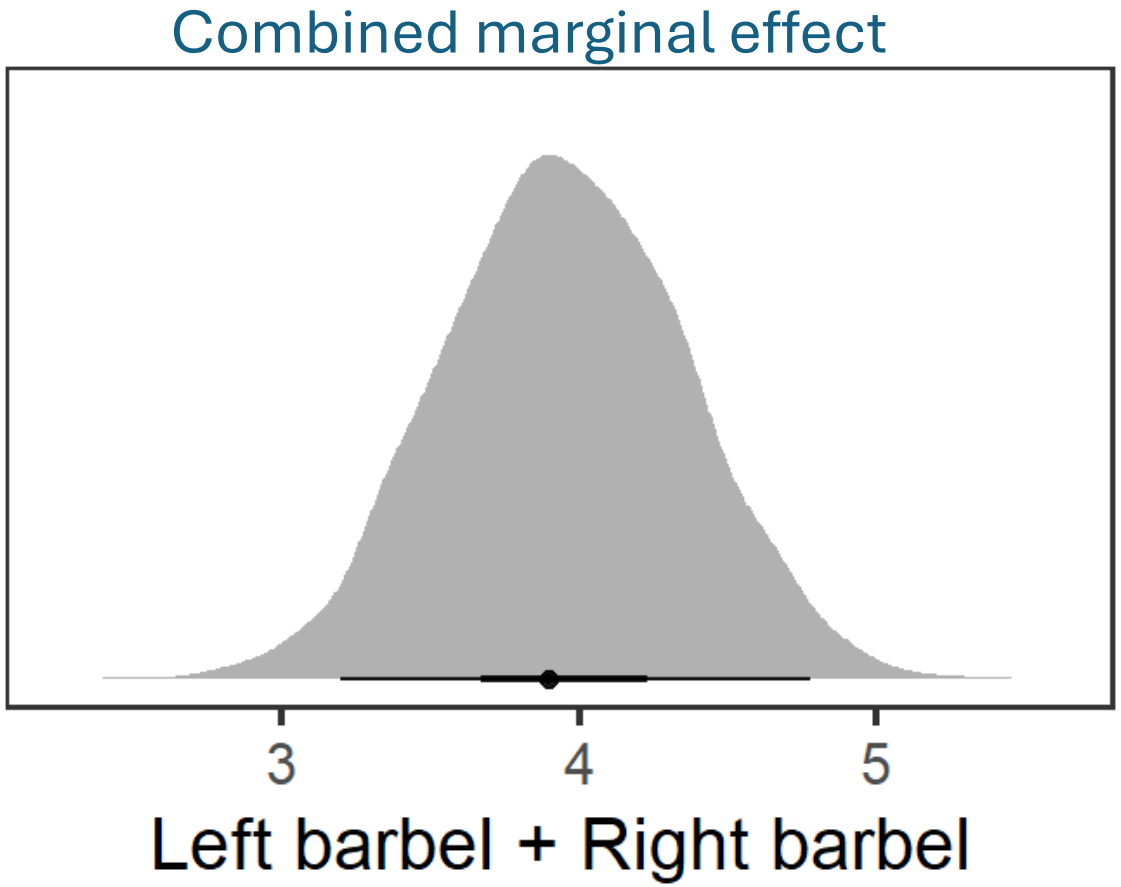
As a result of including both, the posterior distribution for these two slope parameters is very highly correlated

- When the left barbel slope is **large**, then the right barbel slope must be **small**



Correlated barbels

- If you include both in the model, there will be a practically infinite number of combinations of left barbel and right barbel that produce the same predictions
- The posterior ends up reporting the very large range of slope combinations whose sum reflects the actual association
- The posterior here does exactly that

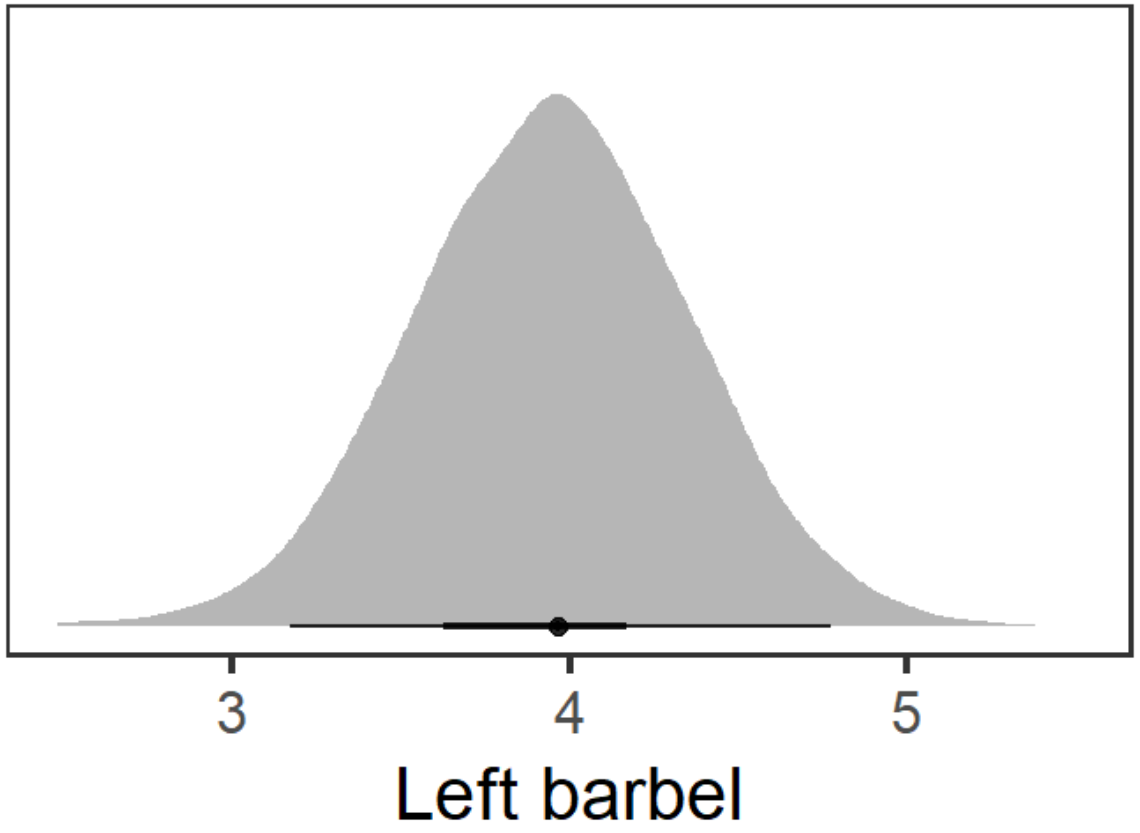


Correlated barbels

- What happens if we only fit one of the barbels?

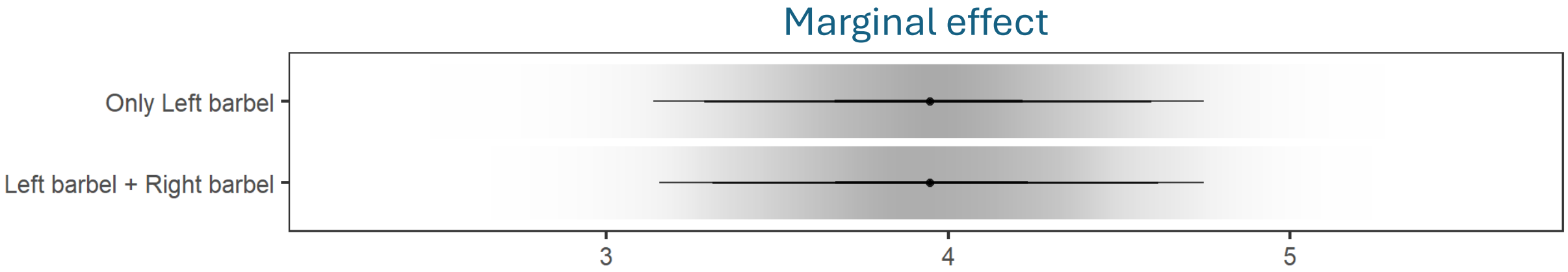
Parameter	Estimate	Est.Error	Q2.5	Q97.5
Left barbel	3.94	0.41	3.14	4.75

Marginal effect



Correlated barbels

- And if we were to predict from both of these models we would get almost the same result



Correlated considerations

The basic takeaway is only: when two predictor variables are very strongly correlated (*conditional on other variables in the model*), including both in a model may lead to confusion

- If you are just interested in prediction, the model will work well
- It just doesn't make any claims about which barbel is more important

Considering the causal structure of your own data is a very important step for including any variables in your own analyses and interpreting their associated output

Coding examples…